



LEARNLYTICS

02
PHASE

Excel Google Sheets

For Data Professionals

Analyze Smarter.
Decide Better.



Organize
Data



Analyze
Insights



Visualize
Results



Automate
Tasks



Master Spreadsheets. Unlock Possibilities.

Unit 1

The Foundation: Data Cleaning & Validation

Topics Covered:

- 1.1 Text Functions: TRIM, CLEAN, PROPER, LEN
- 1.2 Extracting with LEFT, RIGHT, MID, FIND
- 1.3 SUBSTITUTE & REPLACE for Text Fixes
- 1.4 Flash Fill for Pattern-Based Extraction
- 1.5 Finding & Removing Duplicate Records
- 1.6 Data Validation: Dropdowns, Date & Number Rules
- 1.7 IFERROR, IFNA & ISERROR for Error Handling
- 1.8 Standardizing Date & Number Formats
- 1.9 Auditing Tools: Trace Precedents & Dependents
- 1.10 Basic Math & Descriptive Stats: SUM, AVERAGE, COUNT



Learnlytics
— .handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

UNIT 1 — The Foundation: Data Cleaning & Validation

This unit covers the essential Excel skills every data analyst needs to clean, validate, and prepare raw data for analysis. A clean dataset is the foundation of every reliable insight. Without it, even the most sophisticated analysis will produce wrong results.

1.1 Text Functions: TRIM, CLEAN, PROPER, LEN

► Definition

Text functions in Excel allow you to inspect, clean, and transform text data stored in cells. These four foundational functions solve the most common text-quality problems found in raw datasets.

► Sub-Topics

- **TRIM** – Removes all extra spaces from a text string, leaving only single spaces between words. Also removes leading and trailing spaces.
- **CLEAN** – Removes non-printable characters (invisible characters that come from system imports, web scrapes, or copy-pastes).
- **PROPER** – Converts text so that the first letter of each word is uppercase and the rest are lowercase (Title Case).
- **LEN** – Returns the number of characters in a text string, including spaces. Useful for validating field lengths.

► Why Important

Raw data is almost never clean. When data is imported from databases, ERP systems, surveys, or web scrapes, it often contains extra spaces, weird invisible characters, or inconsistent capitalization. These tiny flaws cause:

- VLOOKUP or MATCH failures because " Apple " ≠ "Apple"
- Inaccurate COUNT and SUM results
- Duplicate records that appear unique due to formatting differences

Mastering these functions saves hours of manual correction.

► Real-World Analogy

🔍 Think of TRIM as a word processor's auto-correct that removes double spaces. CLEAN is like a scanner that filters out smudges on a scanned document. PROPER is the grammar rule in a school that says every name must be properly capitalized. LEN is like a word count tool that tells you exactly how long a field is.

► Syntax

Function	Syntax
TRIM	=TRIM(text) e.g., =TRIM(A2)

Function	Syntax
CLEAN	=CLEAN(text) e.g., =CLEAN(A2)
PROPER	=PROPER(text) e.g., =PROPER(A2)
LEN	=LEN(text) e.g., =LEN(A2)

► Example

Suppose column A contains customer names from a database export:

```
A2: " john doe " (extra spaces, all lowercase)
=TRIM(A2) → "john doe" (removes leading/trailing spaces)
=PROPER(TRIM(A2)) → "John Doe" (clean and properly capitalized)
=LEN("John Doe") → 8 (counts all characters)
=CLEAN(TRIM(A2)) → removes hidden characters + extra spaces
```

► Industry Use Case

 **Healthcare:** Patient name fields imported from hospital management systems often have inconsistent casing ("JOHN DOE", "john doe", "John DOE"). A data analyst uses PROPER(TRIM()) to standardize all patient names before merging records across departments — preventing duplicate patient profiles.

 **E-commerce:** Product SKU codes pulled from a supplier portal may contain hidden newline characters. CLEAN() is used to strip these before matching SKUs in a lookup table.

1.2 Extracting Text with LEFT, RIGHT, MID, FIND

► Definition

These functions let you extract specific portions of text from a cell — either from the left, right, or middle. FIND helps you locate a character's position so MID and other functions know where to cut.

► Sub-Topics

- LEFT(text, num_chars) – Extracts a specified number of characters from the beginning (left side) of a text string.
- RIGHT(text, num_chars) – Extracts characters from the end (right side) of a text string.
- MID(text, start_num, num_chars) – Extracts characters from the middle, starting at a position you specify.
- FIND(find_text, within_text, [start_num]) – Returns the position number of a specific character or substring. Case-sensitive.
- SEARCH() – Same as FIND but NOT case-sensitive. Useful when you don't know the exact case.

► Why Important

Real-world datasets are full of combined fields — full names in one column, addresses crammed into one cell, or product codes that encode category + size + color together. Analysts must split these into usable parts. These functions are the surgical tools for text dissection.

► Real-World Analogy

✂ Imagine a long serial number printed on a product box: "NYC-2024-LAPTOP-PRO". LEFT gives you the city code ("NYC"), RIGHT gives you the product type ("PRO"), and MID helps you grab the year ("2024"). FIND is like using a ruler to find exactly where each section starts.

► Syntax

Function	Syntax
LEFT	=LEFT(text, num_chars) e.g., =LEFT(A2, 3)
RIGHT	=RIGHT(text, num_chars) e.g., =RIGHT(A2, 5)
MID	=MID(text, start_num, num_chars) e.g., =MID(A2, 5, 4)
FIND	=FIND(find_text, within_text) e.g., =FIND("@", A2)
SEARCH	=SEARCH(find_text, within_text) e.g., =SEARCH("ltd", A2)

► Example

Suppose A2 contains: "John.Doe@company.com"

```
=FIND("@", A2) → 9 (@ is the 9th character)
=LEFT(A2, FIND("@",A2)-1) → "John.Doe" (everything before @)
=MID(A2, FIND("@",A2)+1, 100) → "company.com" (domain part)
```

Splitting a product code "CAT-2024-BLU":

```
=LEFT(A2, 3) → "CAT" (category)
=MID(A2, 5, 4) → "2024" (year)
=RIGHT(A2, 3) → "BLU" (color code)
```

► Industry Use Case

- 📦 Logistics: Shipment IDs like "MUM-20240312-0045" encode city, date, and sequence. Analysts use LEFT, MID, and RIGHT to parse these into separate columns for city-wise performance reporting.
- 👛 HR/Payroll: Employee IDs like "DEP01-EMP4532" are split using FIND("-") so department codes and employee numbers can be analyzed separately.

1.3 SUBSTITUTE & REPLACE for Text Fixes

► Definition

SUBSTITUTE and REPLACE are the two go-to functions for correcting or modifying text within cells. While they both change text, they work differently: SUBSTITUTE finds and replaces specific text, while REPLACE works based on position.

► Sub-Topics

- **SUBSTITUTE(text, old_text, new_text, [instance_num])** – Finds every occurrence of old_text in a string and replaces it with new_text. The optional 4th argument lets you target only the Nth occurrence.
- **REPLACE(old_text, start_num, num_chars, new_text)** – Replaces a specific number of characters at a given position with new text, regardless of what those characters are.
- **Nested SUBSTITUTE** – You can chain SUBSTITUTE functions to make multiple replacements in one formula.

► Why Important

After data imports, you commonly find: unwanted symbols (\$, %, #), inconsistent separators (commas vs periods in numbers), old codes that need updating, or phone numbers with mixed formatting. SUBSTITUTE and REPLACE let you fix these systematically across thousands of rows in seconds.

► Real-World Analogy

🔍 **SUBSTITUTE** is like the Find & Replace feature in a word processor — you say 'find this word and replace it everywhere'. **REPLACE** is like correcting a typo at a specific position on a typewritten form — you know exactly where the error is, so you just overwrite those specific characters.

► Syntax

Function	Syntax
SUBSTITUTE	<code>=SUBSTITUTE(text, old_text, new_text, [instance])</code> e.g., <code>=SUBSTITUTE(A2, "-", "")</code>
REPLACE	<code>=REPLACE(old_text, start_num, num_chars, new_text)</code> e.g., <code>=REPLACE(A2, 1, 2, "IN")</code>

LEARN SMART. ANALYZE BETTER. GROW FASTER.

► Example

Phone number cleanup — A2 contains: "(022) 555-6789"

```
=SUBSTITUTE(SUBSTITUTE(SUBSTITUTE(A2, "(", ""), ")", ""), "-", "")  
→ "022 5556789" (removes brackets and hyphens)
```

Updating old product codes — A2 contains: "PRD-OLD-001"

```
=SUBSTITUTE(A2, "OLD", "NEW") → "PRD-NEW-001"
```

Replace only the first occurrence of a comma:

```
=SUBSTITUTE(A2, ",", ";", 1) → replaces only the 1st comma with semicolon
```

REPLACE example — fix a 2-digit country code at the start:

```
=REPLACE(A2, 1, 2, "US") → replaces first 2 characters with "US"
```

► Industry Use Case

 **Banking:** Transaction data exported from core banking systems often shows amounts with currency symbols ("₹1,23,456"). Analysts use nested SUBSTITUTE to remove "₹" and ",", before converting text to numbers for calculation.

 **Telecom:** Customer databases may have phone numbers in formats like "(91)98XXXXXXX", "+91-98XXXXXXX", and "0098XXXXXXX" simultaneously. SUBSTITUTE normalizes all variants to a clean 10-digit format.

1.4 Flash Fill for Pattern-Based Extraction

► Definition

Flash Fill is an intelligent Excel feature (introduced in Excel 2013) that automatically detects the pattern you are following and fills in the remaining cells accordingly — without any formula. It learns by example.

► Sub-Topics

- **How to Trigger:** Start typing the desired output in the column next to your data. After 1–2 examples, press Ctrl + E, or go to Data tab → Flash Fill.
- **Pattern Recognition:** Flash Fill can detect splitting, combining, reformatting, case changes, and extraction patterns from your examples.
- **Limitations:** Flash Fill works best with consistent patterns. It can produce errors with irregular or mixed data. The output is static (values, not formulas) — it does not update if the source data changes.
- **Flash Fill vs. Formulas:** Flash Fill is faster for one-time tasks; formulas are better for dynamic, ongoing data.

► Why Important

Flash Fill dramatically speeds up data preparation tasks that would otherwise require complex formulas or manual work. For a data analyst working on a time-sensitive project, being able to split names, reformat dates, or extract codes in seconds without writing a single formula is a significant productivity boost.

► Real-World Analogy

 Flash Fill is like a smart autocomplete on your phone — you start typing a pattern and it predicts the rest. Or imagine training a new assistant: you show them the first two examples of how to reformat a report, and they immediately understand the rule and complete the entire task.

► Syntax

Flash Fill has no formula syntax — it is triggered by:

Keyboard Shortcut: Ctrl + E

Ribbon: Data Tab → Data Tools Group → Flash Fill

Process: Type 1–2 examples of the desired output in adjacent column → Press Ctrl+E

► Example

Column A has full names ("Rahul Sharma", "Priya Patel"). You want first names in column B:

B2: Type "Rahul" (manually)

B3: Type "Priya" (manually)

Press Ctrl+E → Excel fills B4, B5, B6... with first names automatically

Another example — reformatting dates:

A2: "20240315" → B2: type "15-Mar-2024" → Ctrl+E fills the rest

► Industry Use Case



Market Research: Survey data exports often contain full names in one column. Analysts use Flash Fill to instantly separate first name, last name, and title into separate columns for demographic analysis.



Manufacturing: Part numbers like "BOLT-M8-SS-50MM" need to be split into separate attribute columns (type, size, material, length). Flash Fill handles this in under 10 seconds for thousands of rows.

1.5 Finding & Removing Duplicate Records

► Definition

Duplicate records occur when the same data entry appears more than once in a dataset. Excel provides multiple tools to identify, highlight, and remove these duplicates to ensure data integrity.

► Sub-Topics

- **Highlight Duplicates:** Home → Conditional Formatting → Highlight Cells Rules → Duplicate Values. Visually marks all duplicate entries in a selected column or range.
- **Remove Duplicates Tool:** Data Tab → Remove Duplicates. Lets you specify which columns to check and removes rows where all selected columns match.
- **COUNTIF for Detection:** =COUNTIF(\$A\$2:\$A\$100, A2) > 1 returns TRUE for any value that appears more than once. Helps build a helper column to filter duplicates.
- **Unique Values with Formulas:** In Excel 365/2019, =UNIQUE(range) returns only distinct values from a list.
- **Partial Duplicates:** Sometimes rows are duplicates based on a combination of columns (e.g., same Name + Email). The Remove Duplicates tool handles this by letting you select multiple columns.

► Why Important

Duplicates silently destroy the accuracy of your analysis. If a customer appears twice in a sales file, your revenue report double-counts their orders. If a product is listed twice in inventory, stock

levels appear inflated. Removing duplicates is a non-negotiable step in any data cleaning workflow.

► Real-World Analogy

 Imagine voting in an election and someone votes twice. The result is wrong because of the duplicate vote. Duplicate records in data work the same way — they skew totals, averages, and counts. Finding and removing them is like verifying the voter roll to ensure each person appears only once.

► Syntax

Function	Syntax
COUNTIF	=COUNTIF(range, criteria) e.g., =COUNTIF(\$A\$2:\$A\$100, A2)
UNIQUE	=UNIQUE(array, [by_col], [exactly_once]) e.g., =UNIQUE(A2:A100)

Remove Duplicates (via menu): Data Tab → Remove Duplicates → Select columns → OK

► Example

Dataset of 500 customer orders. Column A has Customer_Email:

```
=COUNTIF($A$2:$A$501, A2) → if result > 1, that email is a duplicate  
=IF(COUNTIF($A$2:$A$501, A2)>1, "DUPLICATE", "UNIQUE")
```

Using Remove Duplicates: Select the data range → Data → Remove Duplicates → Check 'Email' and 'Order_ID' → Click OK → Excel reports "X duplicates removed, Y unique values remain".

► Industry Use Case

 Digital Marketing: Email campaign lists collected from multiple lead sources (website forms, events, social media) are merged and then deduplicated using the Remove Duplicates tool on the Email column — preventing customers from receiving the same campaign email multiple times.

 Finance: End-of-day transaction files from payment gateways sometimes contain retry transactions that are exact duplicates. Analysts use COUNTIF across Transaction_ID + Amount + Timestamp columns to flag and remove these before reconciliation.

1.6 Data Validation: Dropdowns, Date & Number Rules

► Definition

Data Validation is an Excel feature that restricts what type of data can be entered into a cell. It acts as a quality gate at the point of data entry — preventing errors before they occur, rather than cleaning them up after.

► Sub-Topics

- **Dropdown Lists:** Create a list of allowable values. Users can only select from the list, preventing typos or invalid entries. Found under: Data → Data Validation → Allow: List.
- **Number Rules:** Restrict entries to whole numbers or decimals within a specific range (e.g., quantity must be between 1 and 999).
- **Date Rules:** Ensure date entries fall within a valid range (e.g., order date must be between 01-Jan-2020 and today).
- **Text Length:** Limit entries to a certain number of characters (e.g., PIN code must be exactly 6 digits).
- **Custom Formula:** Use any Excel formula as a validation rule (e.g., =A2>B2 to ensure end date is after start date).
- **Input Message & Error Alert:** You can add a tooltip that appears when a cell is selected (guidance) and an error message that appears when invalid data is entered.

► Why Important

Data validation enforces consistency at the source. It is the difference between building a clean dataset and dealing with "garbage in, garbage out" problems. For analysts who share templates with business teams, validation ensures the data they receive back is usable.

► Real-World Analogy

 Data Validation is like a customs checkpoint at an airport. Only passengers with valid passports and correct documentation can pass. Similarly, data validation only allows correctly formatted, in-range values to enter the spreadsheet — everything else gets rejected with an explanation.

► Syntax

Data Validation is accessed via menu: Data Tab → Data Validation → Data Validation...

Function	Syntax
Dropdown	Allow: List → Source: =Sheet2!\$A\$1:\$A\$10 or "Yes,No,Maybe"
Number Range	Allow: Whole Number → Data: between → Minimum: 1, Maximum: 100
Date Range	Allow: Date → Data: greater than → Start date: =TODAY()-365
Text Length	Allow: Text Length → Data: equal to → Length: 10
Custom	Allow: Custom → Formula: =AND(A2>0, A2<>"")

► Example

Sales form where Region must be one of: North, South, East, West

Select column B → Data → Data Validation → Allow: List
Source: "North, South, East, West" → OK
Now column B shows a dropdown; any typed value not in the list shows an error.

Order quantity must be between 1 and 500:

Allow: Whole Number → Between → Min: 1 → Max: 500
Error Alert: "Quantity must be between 1 and 500"

► Industry Use Case

 **Pharmaceuticals:** Clinical trial data entry forms use Data Validation extensively — patient age must be between 18 and 75, dosage must be a decimal between 0.5 and 10, and treatment arm must be selected from a predefined dropdown. This prevents invalid trial data from corrupting research results.

 **Construction:** Project cost estimation templates shared with site engineers use number validation to ensure material quantities are positive numbers and date validation to confirm that project end dates are always after start dates.

1.7 IFERROR, IFNA & ISERROR for Error Handling

► Definition

Error handling functions let you detect and gracefully manage formula errors in Excel. Instead of displaying ugly error codes like #N/A, #DIV/0!, or #VALUE!, you can display a custom message, a zero, or a blank cell.

► Sub-Topics

- **IFERROR(value, value_if_error)** – If the formula produces any error, return the alternative value instead. This is the most commonly used error handler. Catches ALL error types.
- **IFNA(value, value_if_na)** – Only catches the #N/A error. This is more precise than IFERROR — useful when you want other errors (like #DIV/0!) to show normally so you can spot real formula problems.
- **ISERROR(value)** – Returns TRUE if the value is any error, FALSE otherwise. Used inside IF statements for more complex conditional logic.
- **ISERR(value)** – Like ISERROR but excludes #N/A errors.
- **Common Excel Errors:** #N/A (not found), #DIV/0! (divide by zero), #VALUE! (wrong data type), #REF! (invalid cell reference), #NAME? (formula typo).

► Why Important

Errors in formulas break calculations, create visual clutter, and confuse end-users. In analytical dashboards and reports shared with non-technical stakeholders, showing #N/A or #DIV/0! looks unprofessional and can cause panic. Error handling makes your work robust and presentation-ready.

► Real-World Analogy

🛡️ IFERROR is like a safety net under a tightrope walker — if something goes wrong, instead of crashing, you have a soft landing. IFNA is a more targeted safety net that only catches one specific type of fall ("not found"), letting other issues surface so you can fix them properly.

► Syntax

Function	Syntax
IFERROR	<code>=IFERROR(value, value_if_error)</code> e.g., <code>=IFERROR(VLOOKUP(A2,D:E,2,0), "Not Found")</code>
IFNA	<code>=IFNA(value, value_if_na)</code> e.g., <code>=IFNA(VLOOKUP(A2,D:E,2,0), "Not Found")</code>
ISERROR	<code>=IF(ISERROR(formula), alt_value, formula)</code> e.g., <code>=IF(ISERROR(A2/B2), 0, A2/B2)</code>
ISERR	<code>=IF(ISERR(value), "Error", value)</code>

► Example

VLOOKUP on a customer table where some customers may not exist:

Without error handling: `=VLOOKUP(A2, CustomerDB, 2, 0)`
→ Returns #N/A if customer not found — breaks the report
With IFERROR: `=IFERROR(VLOOKUP(A2, CustomerDB, 2, 0), "New Customer")`
→ Returns "New Customer" gracefully if not found

Division with possible zero denominators (profit margin):

`=IFERROR(B2/C2, 0)` or `=IF(C2=0, 0, B2/C2)`
→ Shows 0 instead of #DIV/0! when revenue is zero

► Industry Use Case

📊 Retail Analytics: Sales dashboards use IFERROR around VLOOKUP formulas that match product IDs to product names. When new products are added mid-month and not yet in the master table, IFERROR displays "New Product" instead of crashing the entire report.

📈 Investment Banking: Financial models use IFERROR(formula, 0) for ratio calculations (P/E ratio, ROE) where the denominator can be negative or zero in early-stage company analysis — ensuring the model runs without breaking.

1.8 Standardizing Date & Number Formats

► Definition

Standardizing formats means converting dates and numbers stored in various inconsistent formats into a single, consistent format that Excel can correctly interpret, calculate with, and display. This is one of the most critical — and trickiest — data cleaning tasks.

► Sub-Topics

- **Date as Text Problem:** Excel sometimes imports dates as text strings ("15/03/2024" stored as text, not as a real date). These cannot be used in date arithmetic or filtered correctly.
- **DATEVALUE(date_text)** – Converts a date stored as text into an Excel serial number (a real date) that can be formatted and calculated.
- **DATE(year, month, day)** – Builds a real date from separate year, month, and day components. Very useful when date parts are in separate columns.
- **TEXT(value, format_text)** – Converts a number or date into text in a specific display format. e.g., =TEXT(A2, "DD-MMM-YYYY") → "15-Mar-2024".
- **Number Formats:** Custom number formats (Ctrl+1) control how numbers display without changing the underlying value. e.g., #,##0.00 shows thousands separators with 2 decimal places.
- **NUMBERVALUE(text, decimal_sep, group_sep)** – Converts numbers stored as text (with specific separators) into actual numbers.
- **Cell Format vs. Stored Value:** A key concept — formatting only changes how a value looks, not its actual value. The stored value must be a real date or number for calculations to work.

► Why Important

Date and number format mismatches are the #1 cause of calculation errors in analytical work. A date stored as text cannot be sorted correctly, cannot be used in DATEDIF or aging calculations, and will not respond to date filters. Standardizing these is foundational to any analysis.

► Real-World Analogy

 Imagine receiving invoices from 5 different vendors. One writes dates as "15 March 2024", another as "3/15/2024", another as "2024-03-15". Before you can sort them by date or calculate payment due dates, you must convert all of them to the same format. That is exactly what date standardization does in Excel.

► Syntax

Function	Syntax
DATEVALUE	=DATEVALUE(date_text) e.g., =DATEVALUE("15-Mar-2024")
DATE	=DATE(year, month, day) e.g., =DATE(2024, 3, 15)
TEXT	=TEXT(value, format_text) e.g., =TEXT(A2, "DD/MM/YYYY")
YEAR/MONTH/DAY	=YEAR(A2) / =MONTH(A2) / =DAY(A2)
NUMBERVALUE	=NUMBERVALUE(text, dec_sep, grp_sep) e.g., =NUMBERVALUE(A2, ".", ",", "")

► Example

Column A has dates as text: "2024/03/15" — need DD-MMM-YYYY format:

```
=DATEVALUE(A2) → converts text to Excel date serial number  
Then format cell as: DD-MMM-YYYY → displays as "15-Mar-2024"
```

Dates split into 3 columns (Year: C, Month: D, Day: E):

```
=DATE(C2, D2, E2) → combines into a single real date value
```

Number stored as text "1,23,456.00" (Indian format):

```
=NUMBERVALUE(A2, ".", ",") → converts to real number 123456
```

► Industry Use Case

 Supply Chain: Delivery logs exported from SAP systems show dates in YYYYMMDD format ("20240315"). Analysts use =DATE(LEFT(A2,4), MID(A2,5,2), RIGHT(A2,2)) to convert these into real Excel dates for lead time analysis.

 Multinational Reporting: Financial data consolidated from European and US subsidiaries has numbers formatted differently (1.234,56 vs 1,234.56). NUMBERVALUE with the correct separator parameters standardizes both formats before aggregation.

1.9 Auditing Tools: Trace Precedents & Dependents

► Definition

Excel's Auditing Tools are visual tools that help you understand how formulas are connected — which cells a formula draws data from (precedents) and which cells depend on a particular cell (dependents). They are essential for understanding, debugging, and verifying complex spreadsheet models.

► Sub-Topics

- Trace Precedents: Draws blue arrows FROM the cells that feed data INTO the selected formula cell. Helps you verify a formula is using the correct input cells.
- Trace Dependents: Draws blue arrows TO all cells that USE the selected cell in their formulas. Helps you understand the impact of changing a value.
- Remove Arrows: Clears all the audit arrows from the screen.
- Show Formulas (Ctrl + `): Switches the entire sheet to display formulas instead of values — useful for a quick overview of all formula logic.
- Evaluate Formula: Steps through a formula one calculation at a time, showing the intermediate result at each step. Found under Formulas → Formula Auditing → Evaluate Formula.
- Error Checking: Formulas → Error Checking automatically scans the worksheet for formula errors and inconsistencies.
- Watch Window: Lets you monitor specific cells' values in a floating window even when you scroll away from them.

► Why Important

Complex financial models and analytical reports can have hundreds of interconnected formulas. Without auditing tools, finding the source of an error is like searching for a short circuit in a massive wiring board. These tools make the invisible visible — showing you exactly how data flows through your spreadsheet.

► Real-World Analogy

💡 Trace Precedents is like tracing an electrical wire back to its source — you follow the wire from the light bulb back to find which switch controls it. Trace Dependents is the opposite — starting at the switch and seeing which light bulbs it controls. Evaluate Formula is like a slow-motion replay that shows you each step of the calculation.

► Syntax

All Auditing tools are in the Ribbon (no formula syntax):

```
Formulas Tab → Formula Auditing Group
→ Trace Precedents (shortcut: Alt + T + U + T)
→ Trace Dependents (shortcut: Alt + T + U + D)
→ Remove Arrows (removes audit arrows)
→ Evaluate Formula (steps through calculation)
→ Error Checking (scans for errors)
→ Watch Window (monitor remote cells)
Show Formulas shortcut: Ctrl + ` (backtick key)
```

► Example

You have a Total Revenue cell (G50) with formula: =SUM(G2:G49)

```
Select G50 → Formulas → Trace Precedents
→ Blue arrows appear from G2:G49 pointing to G50
→ Confirms the SUM range is correct
```

You have an Assumptions cell (B5) with a tax rate. You want to know what will change if you update it:

```
Select B5 → Formulas → Trace Dependents
→ Arrows show all cells that use B5 in their formulas
→ You can now see the full impact before making changes
```

► Industry Use Case

💰 Investment Banking: Financial models for mergers & acquisitions (M&A) have highly complex cell dependencies (revenue → EBITDA → valuation → offer price). Auditors use Trace Precedents to verify that valuation cells are correctly linked to the right revenue assumptions before a deal closes.

📊 FP&A (Finance): Before presenting a budget model to the CFO, analysts use Evaluate Formula to step through complex nested formulas (e.g., bonuses that depend on revenue tiers) to ensure every calculation is correct.

1.10 Basic Math & Descriptive Stats: SUM, AVERAGE, COUNT

► Definition

These foundational statistical functions form the basis of all quantitative analysis in Excel. They summarize numerical data by computing totals, central tendencies, frequencies, and spread — giving the analyst a quick profile of their dataset.

► Sub-Topics

- SUM(range) – Adds all numbers in a range. Ignores text and blank cells.
- SUMIF(range, criteria, sum_range) – Sums only the values that meet a condition.
- SUMIFS – Like SUMIF but supports multiple conditions simultaneously.
- AVERAGE(range) – Calculates the arithmetic mean (sum ÷ count).
- AVERAGEIF / AVERAGEIFS – Calculates average for cells meeting one or more conditions.
- COUNT(range) – Counts cells containing numbers only.
- COUNTA(range) – Counts all non-empty cells (numbers, text, dates).
- COUNTBLANK(range) – Counts empty/blank cells. Essential for data completeness checks.
- COUNTIF(range, criteria) – Counts cells matching a single condition.
- COUNTIFS – Counts cells meeting multiple conditions.
- MIN / MAX – Returns the smallest / largest value in a range.
- MEDIAN – Returns the middle value when data is sorted. Less sensitive to outliers than AVERAGE.
- MODE – Returns the most frequently occurring value.
- STDEV / STDEVP – Standard deviation for a sample / entire population. Measures data spread.
- VAR / VARP – Variance (square of standard deviation). Another spread measure.
- LARGE(range, k) / SMALL(range, k) – Returns the k-th largest / smallest value.

► Why Important

Before any sophisticated analysis, a data analyst must understand their data. Descriptive statistics answer: How big is the total? What is the typical value? How spread out is the data? Are there any obvious outliers? These 6 questions are answered by SUM, AVERAGE, MEDIAN, COUNT, MIN/MAX, and STDEV — making them the analyst's first tools on any new dataset.

► Real-World Analogy

 Think of descriptive statistics as the nutrition label on a food package. It doesn't tell you everything about the food, but it gives you the key summary facts: total calories (SUM), average serving size (AVERAGE), range from lowest to highest nutrient (MIN/MAX), and how variable the ingredients are (STDEV). That quick summary guides your decisions.

► Syntax

Function	Syntax
SUM	=SUM(range) e.g., =SUM(B2:B100)
SUMIF	=SUMIF(range, criteria, sum_range) e.g., =SUMIF(A:A,"North",B:B)
SUMIFS	=SUMIFS(sum_rng, crit_rng1, crit1, ...) e.g., =SUMIFS(C:C,A:A,"North",B:B,">1000")
AVERAGE	=AVERAGE(range) e.g., =AVERAGE(B2:B100)
AVERAGEIF	=AVERAGEIF(range, criteria, avg_range)
COUNT	=COUNT(range) – numbers only
COUNTA	=COUNTA(range) – all non-empty
COUNTBLANK	=COUNTBLANK(range) – empty cells only
COUNTIF	=COUNTIF(range, criteria) e.g., =COUNTIF(A:A,"Active")
COUNTIFS	=COUNTIFS(range1, crit1, range2, crit2)
MIN / MAX	=MIN(range) / =MAX(range)
MEDIAN	=MEDIAN(range)
STDEV	=STDEV(range) – sample std deviation
LARGE/SMALL	=LARGE(range, k) / =SMALL(range, k) e.g., =LARGE(B:B,3)

► Example

Sales dataset: Column A = Region, Column B = Sales Amount, 500 rows

```

=SUM(B2:B501) → Total sales across all regions
=SUMIF(A2:A501, "North", B2:B501) → Total sales for North region only
=AVERAGE(B2:B501) → Average sale per transaction
=MEDIAN(B2:B501) → Middle value (less skewed by outliers)
=COUNT(B2:B501) → How many numeric sales entries exist
=COUNTIF(A2:A501, "North") → How many North region transactions
=COUNTBLANK(B2:B501) → How many rows have missing sales data
=MAX(B2:B501) → Highest single sale amount
=MIN(B2:B501) → Lowest single sale amount
=STDEV(B2:B501) → How much sales vary from the average
=LARGE(B2:B501, 3) → 3rd highest sale amount

```

► Industry Use Case

 **Retail:** At the end of each month, a retail analyst uses SUMIFS to calculate region-wise and category-wise revenue (e.g., North + Electronics), COUNTIFS to count the number of transactions in each segment, and AVERAGE + STDEV to profile the typical basket size and its variability — all before building a dashboard.

 **Healthcare Analytics:** Hospital administrators use AVERAGEIF to calculate the average length of stay (LOS) by department, COUNTIF to count readmissions by diagnosis code, and MEDIAN instead of AVERAGE for billing amounts (since a few extremely high bills would skew the AVERAGE, making MEDIAN a more accurate representation of the typical patient bill).

UNIT 1 — Interview Base Unit Tests

The Foundation: Data Cleaning & Validation

Assessment Structure

- **Level 1: Warmup Test** 10 Basic Questions (3 MCQ · 1 Descriptive · 2 Scenario · 2 Coding · 2 DB Scenario)
- **Level 2: Interviewer Test** 10 Intermediate Questions (2 MCQ · 2 Descriptive · 2 Scenario · 2 Coding · 2 DB Scenario)

Instructions

- Answer all questions within each level.
- For Descriptive, Coding, and Scenario questions — write your answer in the box provided.
- MCQ correct answers are compiled in the Answer Key section at the end of this paper.
- Coding questions should include the complete Excel formula(s) with explanations.
- Database Scenario questions require formulas referencing the given column structure.

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Level 1: Warmup Test | 10 Basic Questions

Topics: TRIM, CLEAN, PROPER, LEN, LEFT, RIGHT, MID, FIND, Flash Fill, Duplicates, Data Validation, Error Handling, Date Formats, Descriptive Stats

MCQ Questions (3)

Circle or write the letter of the correct answer.

Q1. [MCQ]

Which Excel function removes leading, trailing, and extra internal spaces from a text string?

- A) CLEAN
- B) TRIM
- C) PROPER
- D) LEN

Q2. [MCQ]

You apply =COUNTIF(\$A\$2:\$A\$500, A2) to each row in a dataset. A result of 3 for a particular row means:

- A) The cell contains 3 characters
- B) The value in that row appears 3 times in the range — a duplicate
- C) Three cells are blank above it
- D) The formula has 3 errors

Q3. [MCQ]

Which Excel error code appears when VLOOKUP cannot find the lookup value in the lookup range?

- A) #VALUE!
- B) #DIV/0!
- C) #N/A
- D) #REF!

Descriptive Question (1)

Q4. [Descriptive]

Explain the purpose and difference between the TRIM and CLEAN functions in Excel. In what type of data situation would you need to use both together?

➡ Answer:

Scenario-Based Descriptive Questions (2)

Q5. [Scenario — Descriptive]

Scenario: You receive a customer master file from the CRM team. The 'Full Name' column contains entries such as: " JOHN DOE ", "mary smith", "PRIYA PATEL", and " Rahul Sharma ". The data needs to be standardized before it can be loaded into the reporting database.

Describe your step-by-step approach — which functions would you apply in what order, and why? What risks remain even after cleaning?

➡ Answer:

Q6. [Scenario — Descriptive]

Scenario: A sales team submits weekly transaction data using a shared Excel template. The 'Region' column frequently has inconsistent entries ("north", "SOUTH", "East ", "west "). The 'Quantity' column sometimes contains values like -5 or 10,000 that are clearly errors.

How would you use Data Validation rules to prevent these errors at the point of entry? What type of validation would you apply to each column, and what error message would you show?

➡ Answer:

Coding Questions (2)

Q7. [Coding]

Column A contains product codes in the format "CAT-2024-BLU" (Category-Year-Color). Write separate Excel formulas to extract:

- (a) The Category code (first 3 characters)
- (b) The Year (characters 5 to 8)
- (c) The Color code (last 3 characters)
- (d) A validation formula that checks the total length is exactly 11 characters

Answer:

Q8. [Coding]

Column A has phone numbers in mixed formats: "(022) 555-1234", "+91-98765-43210", "0091 9876543210". Write a nested SUBSTITUTE formula to clean all phone numbers by removing: brackets (), plus signs (+), hyphens (-), and extra spaces. Show the formula step by step.

Answer:

Database Scenario — Coding Questions (2)

The following Excel table structure is used for Questions 9 and 10:

```
Table: CustomerOrders
A: CustomerID | B: CustomerEmail | C: Region | D: OrderDate | E: OrderAmount
Rows 2-501 contain 500 orders from various customers and regions.
```

Q9. [DB Scenario — Coding]

Using the CustomerOrders table above, write Excel formulas to answer the following business questions:

- (a) How many total orders were placed by customers in the 'North' region?
- (b) What is the total OrderAmount for all orders where the amount is greater than ₹5,000?
- (c) How many cells in column E (OrderAmount) are blank or missing?

- (d) What is the average order amount for 'South' region customers?

➡ Answer:

Q10. [DB Scenario — Coding]

The CustomerOrders table has been imported from an external system. CustomerEmail in column B may contain duplicate entries (the same customer ordering multiple times). Write formulas to:

- (a) Flag each row as 'DUPLICATE' or 'UNIQUE' based on whether the CustomerEmail appears more than once
- (b) Count how many distinct (unique) customers are in the dataset
- (c) Find the email address that appears most frequently (most orders placed)

➡ Answer:

Learnlytics
— .handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Level 2: Interviewer Test | 10 Intermediate Questions

Topics: SUBSTITUTE, REPLACE, IFERROR, IFNA, Date Standardization, Data Validation (advanced), SUMIFS, COUNTIFS, Formula Auditing

MCQ Questions (2)

Q1. [MCQ]

You wrap a VLOOKUP in IFERROR but realize it is hiding a #VALUE! error caused by a data type mismatch — a real bug you need to see. Which function should you have used instead to only suppress #N/A errors?

- A) IFERROR
- B) IFNA
- C) ISERROR
- D) ISERR

Q2. [MCQ]

A date value "20240315" is stored as text in cell A2. Which formula correctly converts it into a real Excel date using date part extraction?

- A) =DATEVALUE(A2)
- B) =DATE(LEFT(A2,4), MID(A2,5,2), RIGHT(A2,2))
- C) =TEXT(A2, "YYYY-MM-DD")
- D) =NUMBERVALUE(A2, ".", ",", ",")

Descriptive Questions (2)

Q3. [Descriptive]

Compare and contrast the SUBSTITUTE and REPLACE functions in Excel. Give a specific real-world example where REPLACE is the better choice and explain why SUBSTITUTE would be less suitable in that scenario.

Answer:

Q4. [Descriptive]

Explain the 'Date as Text' problem in Excel. What are the three different root causes that can create this problem, and what are three different Excel functions used to resolve it? When would each fix be most appropriate?

➡ Answer:

🔗 Scenario-Based Descriptive Questions (2)

Q5. [Scenario — Descriptive]

Scenario: A multinational company consolidates monthly transaction files from 4 regional offices. The date column contains three different formats: 'Region A: 15-Mar-2024', 'Region B: 3/15/2024', 'Region C: 20240315', and 'Region D: March 15, 2024'. All need to be converted to a single consistent DD/MM/YYYY format for the consolidated analysis.

Describe your complete approach — one formula strategy per region — to convert each format into a real Excel date. Which region presents the biggest challenge and why?

➡ Answer:

Q6. [Scenario — Descriptive]

Scenario: You are designing a shared data entry template for 50 field sales representatives across 4 regions. The template must ensure: (1) Region can only be 'North', 'South', 'East', or 'West'. (2) Quantity must be a whole number between 1 and 500. (3) Order Date cannot be in the future. (4) Product Category must be selected from a lookup list in another sheet. (5) Customer PAN number must be exactly 10 characters.

Design the complete Data Validation configuration for each of these 5 requirements. Include the validation type, settings, and appropriate error alert messages.

➡ Answer:

Coding Questions (2)

Q7. [Coding]

Column A contains employee IDs in the format 'DEP01-EMP4532' (5-char dept code, hyphen, 3-char EMP prefix, 4-digit number). Write formulas to:

- (a) Extract the department code (e.g., 'DEP01')
- (b) Extract the numeric employee number (e.g., '4532')
- (c) Validate the format: check it is exactly 13 characters with a hyphen at position 6
- (d) Build a new ID combining only the department code and number, removing the 'EMP' prefix: e.g., 'DEP01-4532'

➡ Answer:

Q8. [Coding]

A sales dataset has columns: A=OrderID, B=Region, C=ProductCategory, D=Quantity, E=UnitPrice, F=Revenue (=D*E). Write formulas using SUMIFS and COUNTIFS to:

- (a) Total Revenue for the 'North' region AND 'Electronics' category
- (b) Count of orders where Quantity > 10 AND Revenue > ₹5,000
- (c) Average Revenue for the 'South' region (using AVERAGEIF)
- (d) Flag rows where F (Revenue) ≠ D*E — data integrity check formula

➡ Answer:

Database Scenario — Coding Questions (2)

Table: InventoryDB

A: SKU | B: ProductName | C: Category | D: StockQty | E: ReorderLevel | F: UnitPrice

Rows 2-201 contain 200 product records. Some cells in D and F may be blank.

Q9. [DB Scenario — Coding]

Using the InventoryDB table, write formulas to answer these analytical questions:

- (a) Count products where StockQty (D) is below their ReorderLevel (E) — low stock alert
- (b) Calculate total inventory value (StockQty × UnitPrice) for the 'Electronics' category
- (c) Find the 3rd most expensive product's UnitPrice using LARGE
- (d) Count how many products have missing (blank) UnitPrice values — data completeness check
- (e) Wrap part (b) in an IFERROR to return 0 if any calculation errors occur

Answer:

Q10. [DB Scenario — Coding]

A hospital patient database has: A=PatientID, B=Ward, C=AdmissionDate, D=DischargeDate, E=TotalBill. Some DischargeDate cells are blank (patients still admitted). Write formulas to:

- (a) Count patients currently admitted (DischargeDate is blank)
- (b) Calculate average TotalBill for the 'ICU' ward
- (c) Find the maximum TotalBill across all patients
- (d) Flag rows where TotalBill is blank using IF and ISBLANK
- (e) Calculate total revenue from all DISCHARGED patients only (DischargeDate is not blank)

Answer:

UNIT 1 — Interview Base Unit Tests Answer

The Foundation: Data Cleaning & Validation

🌀 Level 1: Warmup Test | 10 Basic Questions

🌀 MCQ Questions

Q1. [MCQ] Which Excel function removes leading, trailing, and extra internal spaces?

Correct Answer: B) TRIM

TRIM removes all leading/trailing spaces and reduces internal multiple spaces to a single space. CLEAN removes non-printable characters (not spaces). PROPER changes text case. LEN counts characters.

Q2. [MCQ] =COUNTIF(\$A\$2:\$A\$500, A2) returns 3. This means:

Correct Answer: B) The value in that row appears 3 times in the range — a duplicate

COUNTIF counts how many times the cell's value appears in the range. A result of 3 means the value is repeated 3 times — indicating duplicates.

Q3. [MCQ] Which error appears when VLOOKUP cannot find the lookup value?

Correct Answer: C) #N/A

#N/A means 'Not Available' — the lookup value was not found in the lookup range. #VALUE! is a data type error; #DIV/0! is a division by zero; #REF! is an invalid cell reference.

📄 Descriptive Question

Q4. [Descriptive] Explain the purpose and difference between TRIM and CLEAN functions.

TRIM: Removes leading spaces, trailing spaces, and reduces internal multiple spaces to a single

space. It does NOT remove non-printable characters.

CLEAN: Removes all non-printable characters (ASCII values 0-31), such as line breaks, tabs,

and other control characters imported from external systems. It does NOT remove extra spaces.

When to use both together:

When data is imported from legacy databases, ERP systems, or CSV exports that contain both

extra spaces AND embedded non-printable characters (e.g., carriage returns or null bytes).

Combined formula: `=TRIM(CLEAN(A2))`

CLEAN runs first to strip non-printable characters, then TRIM cleans up any remaining spaces.

This is the safest approach for imported or CRM data.

Scenario-Based Questions

Q5. [Scenario] Standardize Full Name column: " JOHN DOE ", "mary smith", "PRIYA PATEL", "Rahul Sharma "

Step-by-step approach:

Step 1 – Remove extra spaces with TRIM:

`=TRIM(A2)` → Eliminates leading, trailing, and internal double spaces.

Step 2 – Standardize casing with PROPER:

`=PROPER(TRIM(A2))` → Converts to Title Case (e.g., 'JOHN DOE' → 'John Doe').

Step 3 – Handle non-printable characters (if imported data):

`=PROPER(TRIM(CLEAN(A2)))` → The safest combined formula.

Risks that remain even after cleaning:

- Names like 'McDonald' become 'Mcdonald' (PROPER cannot handle special capitalization).
- Hyphenated names (e.g., 'Mary-Jane') may have wrong casing.
- Abbreviated titles (e.g., 'DR.', 'MR.') get lowercased to 'Dr.', 'Mr.' – usually fine.
- Genuine typos in name spelling are not corrected by any formula.
- Unicode or accented characters may not be handled consistently across systems.

Q6. [Scenario] Use Data Validation to prevent errors in Region and Quantity columns.

Region Column – Inconsistent entries like 'north', 'SOUTH', 'East ', 'west ':

Validation Type: List

Source: North,South,East,West (or a named range with these 4 values)

Error Alert: Stop | Title: 'Invalid Region'

Message: 'Please select a valid region: North, South, East, or West.'

Note: Also pre-clean existing data using =PROPER(TRIM(A2)) before applying validation.

Quantity Column – Values like -5 or 10,000 are clearly errors:

Validation Type: Whole Number

Data: Between | Minimum: 1 | Maximum: 9999 (adjust max based on business rules)

Error Alert: Stop | Title: 'Invalid Quantity'

Message: 'Quantity must be a whole number between 1 and 9999. Negative values and',
'excessively large numbers are not allowed.'

Coding Questions

Q7. [Coding] Extract parts from product code format 'CAT-2024-BLU' in column A.

(a) Category code (first 3 characters):

=LEFT(A2, 3)

Result: 'CAT'

(b) Year (characters 5 to 8):

=MID(A2, 5, 4)

Explanation: Start at position 5 (after 'CAT-'), extract 4 characters.

Result: '2024'

(c) Color code (last 3 characters):

=RIGHT(A2, 3)

Result: 'BLU'

(d) Validation – check total length is exactly 11 characters:

=IF(LEN(A2)=11, "Valid", "Invalid")

Explanation: 'CAT-2024-BLU' = 3+1+4+1+3 = 12 characters.

Wait – let us count: C-A-T = 3, dash = 1, 2-0-2-4 = 4, dash = 1, B-L-U = 3 = 12 total.

=IF(LEN(A2)=12, "Valid", "Invalid") ← Correct length is 12, not 11.

Q8. [Coding] Clean phone numbers using nested SUBSTITUTE to remove () + - and spaces.

Complete nested formula:

```
=SUBSTITUTE(SUBSTITUTE(SUBSTITUTE(SUBSTITUTE(SUBSTITUTE(A2,"(", ""),"),", ""),"+", ""),"-", "")," ", "")
```

Step-by-step breakdown:

Step 1 – Remove opening parenthesis '(':

```
=SUBSTITUTE(A2, "(", "")  
'(022) 555-1234' → '022) 555-1234'
```

Step 2 – Remove closing parenthesis ')':

```
=SUBSTITUTE(Step1, ")", "")  
'022) 555-1234' → '022 555-1234'
```

Step 3 – Remove plus sign '+':

```
=SUBSTITUTE(Step2, "+", "")  
'+91-98765-43210' → '91-98765-43210'
```

Step 4 – Remove hyphens '-':

```
=SUBSTITUTE(Step3, "-", "")  
'022 5551234' → '022 5551234' and '9198765 43210' → '919876543210'
```

Step 5 – Remove all spaces ' ':

```
=SUBSTITUTE(Step4, " ", "")  
Result: '0225551234', '919876543210', '00919876543210'
```

Database Scenario — Coding Questions

Table: CustomerOrders | A: CustomerID | B: CustomerEmail | C: Region | D: OrderDate | E: OrderAmount | Rows 2–501

Q9. [DB Scenario] Write formulas to answer business questions on CustomerOrders.

(a) Total orders placed by customers in the 'North' region:

=COUNTIF(C2:C501, "North")

Counts rows where the Region column equals 'North'.

(b) Total OrderAmount for orders greater than Rs.5,000:

=SUMIF(E2:E501, ">5000", E2:E501)

Sums values in E where E > 5000.

(c) Blank/missing cells in column E (OrderAmount):

=COUNTBLANK(E2:E501)

Counts empty or blank cells in the OrderAmount range.

(d) Average order amount for 'South' region customers:

=AVERAGEIF(C2:C501, "South", E2:E501)

Averages OrderAmount values where Region = 'South'.

Q10. [DB Scenario] Handle duplicate CustomerEmails in column B.

(a) Flag each row as 'DUPLICATE' or 'UNIQUE' based on CustomerEmail frequency:

=IF(COUNTIF(\$B\$2:\$B\$501, B2) > 1, "DUPLICATE", "UNIQUE")

Place in a helper column. Use absolute reference for the range, relative for the cell.

Copy down for all 500 rows.

(b) Count of distinct (unique) customers in the dataset:

=SUMPRODUCT(1 / COUNTIF(B2:B501, B2:B501))

Divides 1 by each email's count, then sums – each unique email contributes exactly 1.

Note: Wrap in IFERROR if any blanks exist:

=SUMPRODUCT(IFERROR(1 / COUNTIF(B2:B501, B2:B501), 0))

(c) Email address that appears most frequently (most orders placed):

=INDEX(B2:B501, MATCH(MAX(COUNTIF(B2:B501, B2:B501)), COUNTIF(B2:B501, B2:B501), 0))

COUNTIF creates a frequency array; MAX finds the highest count;

MATCH locates its position; INDEX retrieves the email at that position.



Learnlytics

.handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Level 2: Interviewer Test | 10 Intermediate Questions

MCQ Questions

Q1. [MCQ] Which function should you use to suppress only #N/A errors (not hide real bugs)?

Correct Answer: B) IFNA

IFNA catches only #N/A errors and returns a custom value. IFERROR catches ALL errors (including #VALUE!, #REF!, etc.), which can mask genuine bugs. ISERROR and ISERR are logical tests, not error-handling wrappers.

Q2. [MCQ] Convert text date '20240315' in A2 to a real Excel date.

Correct Answer: B) =DATE(LEFT(A2,4), MID(A2,5,2), RIGHT(A2,2))

This formula extracts the year (LEFT 4 chars), month (MID from position 5, 2 chars), and day (RIGHT 2 chars), then assembles a real date serial number. DATEVALUE() won't recognize the compact format '20240315' without separators. TEXT() formats existing dates, not text. NUMBERTOVALUE() converts numeric text, not date strings.

Descriptive Questions

Q3. [Descriptive] Compare SUBSTITUTE vs REPLACE. When is REPLACE the better choice?

`SUBSTITUTE(text, old_text, new_text, [instance_num])`

- Replaces a specific TEXT STRING wherever it appears.
- Does not require knowing the position.
- Best when you know WHAT to replace but not WHERE it is.
- Example: `=SUBSTITUTE(A2, "Ltd", "Limited")` replaces 'Ltd' anywhere in the cell.

`REPLACE(old_text, start_num, num_chars, new_text)`

- Replaces characters at a SPECIFIC POSITION for a SPECIFIC LENGTH.
- Does not care about the content at that position.
- Best when you know WHERE to replace but not the exact content.

Real-world example where REPLACE is better:

Employee codes always have a 2-digit department prefix that needs updating:

Old format: 'HR-00123' → New format: 'HRD-00123'

`=REPLACE(A2, 1, 2, "HRD")`

Why SUBSTITUTE fails here: If 'HR' appears elsewhere in the string (e.g., data contains

'HR-CHRM-001'), SUBSTITUTE would incorrectly replace both occurrences.

Q4. [Descriptive] Explain the 'Date as Text' problem — 3 causes and 3 Excel fixes.

The 'Date as Text' Problem:

Excel stores dates as serial numbers (e.g., Jan 1 2024 = 45292). When dates are stored as text strings, date arithmetic, sorting, and filtering all break.

Three Root Causes:

1. Imported data: Dates from external systems (CSV, ERP, APIs) often arrive as text strings in formats Excel doesn't auto-recognize (e.g., '20240315', 'March 15 2024').
2. Apostrophe prefix or text-formatted cells: A leading apostrophe (') forces Excel to treat the cell as text. If the column was pre-formatted as Text before data entry, even valid date formats are stored as strings.
3. Non-standard or ambiguous formats: Dates like '15/03/2024' may be misread as text in regions where MM/DD/YYYY is the default, or vice versa.

Three Excel Functions to Resolve:

1. DATEVALUE(text): Converts a recognizable text date string to a serial number.
Best for: Strings like '15-Mar-2024' or '3/15/2024' that match known formats.
=DATEVALUE(A2)
2. DATE() + LEFT/MID/RIGHT: Manually extracts year, month, day from fixed-format strings.
Best for: Compact strings like '20240315' with no separators.
=DATE(LEFT(A2,4), MID(A2,5,2), RIGHT(A2,2))
3. Text to Columns wizard (Data tab): A bulk, formula-free fix for a whole column.
Best for: Large datasets where formulas would be slow or cumbersome.
Steps: Select column → Data → Text to Columns → Delimited → Finish.

Q5. [Scenario] Convert 4 regional date formats to DD/MM/YYYY.

Region A: '15-Mar-2024' (text, human-readable format)
=TEXT(DATEVALUE(A2), "DD/MM/YYYY")
DATEVALUE recognizes '15-Mar-2024' directly and converts to a serial number.
TEXT then formats it as DD/MM/YYYY.

Region B: '3/15/2024' (US M/D/YYYY format as text)
=TEXT(DATE(RIGHT(A2,4), LEFT(A2,FIND("/",A2)-1),
MID(A2,FIND("/",A2)+1,FIND("/",A2,FIND("/",A2)+1)-FIND("/",A2)-1)), "DD/MM/YYYY")
Or simpler if column format is consistent: =TEXT(DATEVALUE(A2),"DD/MM/YYYY")
Note: Locale-dependent – safer to extract parts explicitly.

Region C: '20240315' (compact YYYYMMDD text – NO separators)
=TEXT(DATE(LEFT(A2,4), MID(A2,5,2), RIGHT(A2,2)), "DD/MM/YYYY")
Extracts year, month, day by fixed character positions.

Region D: 'March 15, 2024' (long-form month name)
=TEXT(DATEVALUE(A2), "DD/MM/YYYY")
DATEVALUE handles spelled-out month names in most English locales.

Biggest challenge: Region C ('20240315')
No separators means DATEVALUE() cannot parse it – manual extraction is mandatory.
Also risky: if the string had any extra characters or spaces, the formula breaks.
A data audit step is recommended before applying the formula.

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Q6. [Scenario] Design Data Validation for 5 requirements in a shared sales template.

Requirement 1 – Region (North/South/East/West only):
Validation Type: List
Source: North,South,East,West
Error Alert: Stop | Title: 'Invalid Region' | Message: 'Select from North, South, East, West only.'

Requirement 2 – Quantity (whole number 1–500):

Validation Type: Whole Number | Data: Between | Min: 1 | Max: 500

Error Alert: Stop | Title: 'Invalid Quantity' | Message: 'Enter a whole number between 1 and 500.'

Requirement 3 – Order Date (cannot be in the future):

Validation Type: Date | Data: Less than or equal to | End Date: =TODAY()

Error Alert: Stop | Title: 'Invalid Date' | Message: 'Order Date cannot be a future date.'

Requirement 4 – Product Category (from lookup list on another sheet):

Validation Type: List

Source: =ProductList!\$A\$2:\$A\$50 (named range 'Categories' is even cleaner)

Error Alert: Stop | Title: 'Invalid Category' | Message: 'Select a category from the approved list.'

Requirement 5 – Customer PAN (exactly 10 characters):

Validation Type: Custom

Formula: =LEN(A2)=10

Error Alert: Stop | Title: 'Invalid PAN' | Message: 'PAN number must be exactly 10 characters.'

Coding Questions

Q7. [Coding] Extract from Employee ID format 'DEP01-EMP4532' in column A.

(a) Extract department code (e.g., 'DEP01' – everything before the hyphen):

=LEFT(A2, FIND("-", A2) - 1)

FIND locates the hyphen; LEFT extracts all characters before it.

Result: 'DEP01'

(b) Extract numeric employee number (e.g., '4532' – last 4 digits):

=RIGHT(A2, 4)

Or more dynamically: =MID(A2, FIND("-", A2) + 4, 4)

Skips the '-EMP' part (3 chars after hyphen) to get the 4-digit number.

Result: '4532'

(c) Validate format – exactly 13 characters with hyphen at position 6:

`=AND(LEN(A2) = 13, MID(A2, 6, 1) = "-")`

Returns TRUE if valid, FALSE if not.

Optional with label: `=IF(AND(LEN(A2)=13, MID(A2,6,1)="-"), "Valid", "Invalid")`

(d) Build new ID combining dept code and number, removing 'EMP' prefix:

`=LEFT(A2, FIND("-", A2)) & RIGHT(A2, 4)`

LEFT up to and including the hyphen, then concatenate the last 4 digits.

Result: 'DEP01-4532'

Q8. [Coding] SUMIFS/COUNTIFS on sales dataset A=OrderID, B=Region, C=Category, D=Qty, E=UnitPrice, F=Revenue.

(a) Total Revenue for 'North' region AND 'Electronics' category:

`=SUMIFS(F2:F501, B2:B501, "North", C2:C501, "Electronics")`

Sums F (Revenue) only where B='North' AND C='Electronics'.

(b) Count of orders where Quantity > 10 AND Revenue > Rs.5,000:

`=COUNTIFS(D2:D501, ">10", F2:F501, ">5000")`

Counts rows meeting BOTH conditions simultaneously.

(c) Average Revenue for 'South' region (using AVERAGEIF):

`=AVERAGEIF(B2:B501, "South", F2:F501)`

Averages F (Revenue) where B (Region) = 'South'.

(d) Flag rows where Revenue (F) does not equal D*E – data integrity check:

`=IF(F2 <> D2 * E2, "DATA ERROR", "OK")`

Place this in a helper column (e.g., G2) and copy down.

Highlights any row where the stored revenue doesn't match Qty x UnitPrice.

Database Scenario — Coding Questions

Table: InventoryDB | A: SKU | B: ProductName | C: Category | D: StockQty | E: ReorderLevel | F: UnitPrice | Rows 2–201

Q9. [DB Scenario] InventoryDB analytical formulas.

(a) Count products where StockQty (D) is below their ReorderLevel (E) – low stock alert:

=SUMPRODUCT((D2:D201 < E2:E201) * 1)

Compares each D value against its corresponding E value row by row.

Returns count of rows where StockQty < ReorderLevel.

(b) Total inventory value (StockQty x UnitPrice) for 'Electronics' category:

=SUMPRODUCT((C2:C201 = "Electronics") * D2:D201 * F2:F201)

Multiplies the Boolean filter by D and F arrays, then sums the results.

(c) 3rd most expensive product's UnitPrice:

=LARGE(F2:F201, 3)

LARGE returns the k-th largest value; k=3 gives the 3rd highest UnitPrice.

(d) Count products with missing (blank) UnitPrice values:

=COUNTBLANK(F2:F201)

Counts all empty or blank cells in the UnitPrice column.

(e) Part (b) wrapped in IFERROR to return 0 on any calculation error:

=IFERROR(SUMPRODUCT((C2:C201="Electronics") * D2:D201 * F2:F201), 0)

If the SUMPRODUCT encounters an error (e.g., text in numeric columns), returns 0.

Table: PatientDB | A: PatientID | B: Ward | C: AdmissionDate | D: DischargeDate | E: TotalBill |
Rows 2–501

Q10. [DB Scenario] Hospital patient database formulas.

(a) Count patients currently admitted (DischargeDate D is blank):

=COUNTBLANK(D2:D501)

Blank DischargeDate means the patient is still admitted.

(b) Average TotalBill for the 'ICU' ward:

=AVERAGEIF(B2:B501, "ICU", E2:E501)

Averages TotalBill (E) where Ward (B) = 'ICU'.

(c) Maximum TotalBill across all patients:

`=MAX(E2:E501)`

Returns the single largest value in the TotalBill column.

(d) Flag rows where TotalBill (E) is blank using IF and ISBLANK:

`=IF(ISBLANK(E2), "Missing Bill", "OK")`

Place in a helper column and copy down all 500 rows.

Returns 'Missing Bill' for empty cells, 'OK' for cells with data.

(e) Total revenue from DISCHARGED patients only (DischargeDate is not blank):

`=SUMIF(D2:D501, "<>", E2:E501)`

SUMIF with '<>' (not equal to blank) sums TotalBill only for discharged patients.

Alternative: `=SUMPRODUCT((D2:D501<>"") * E2:E501)`



LEARN SMART. ANALYZE BETTER. GROW FASTER.